

Q. What is the difference between micro-controller & microprocessor?

### Microprocessor

- 1) heart of a computer system
- 2) It is just a processor. Memory & I/O components have to be connected externally.
- 3) Cost of the entire system increases.
- 4) cannot be used in compact systems & hence inefficient
- 5) Entire Power consumption is high.
- 6) Microprocessors have less no. of registers, hence more operations are memory based.

### Microcontroller

- 1) heart of embedded system
- 2) Microcontroller has external processor along with internal memory & I/O components.
- 3) Cost of the entire system is low.
- 4) can be used in compact systems & hence it is an efficient technique.
- 5) Since external components are low, total power consumption is less.
- 6) Microcontrollers have more no. of registers, hence the programs are easier to write.



②

Monika - 56352

7) Microprocessors are based on von-Neumann model where program & data are stored in same memory module.

8) Mainly used in personal computers.

7) Microcontrollers are based on Harvard architecture where program memory & data memory are separate.

8) Used mainly in washing machine, mp3 players.

Que. 2)

What is the difference between I/O mapped I/O & memory mapped I/O?

⇒ Memory mapped I/O

1) I/O devices (SFRs) are mapped as memory addresses.

2) Access with same instructions as of memory, e.g. MOV, LD/ST.

3) No differentiation between memory location

I/O mapped I/O

1) I/O devices (I/O registers) are mapped as separate I/O addresses (in diff. address space)

2) I/O reg's are accessed with special instr's e.g. IN/OUT.

3) Memory location & I/O refers are differentiated



③

8 IO registers  
except address.

by different buses  
or control signals.

4) IO do not have  
their own address  
space.

4) IO have own  
address space.

5) Some memory leakage.

5) No memory leakage.

6) used when there are  
more no. of I/O  
devices.

6) used when less  
no. of I/O  
devices.

7) e.g. ARM

7) e.g. 8086

Ques 4) What is the difference between  
Fast GPIO & legacy GPIO?

⇒ Fast GPIO

Legacy GPIO

1) Connected to ARM  
core bus (AHB)

1) Connected to peripheral  
bus (APB)

2) works at higher speed  
i.e. core speed

2) works at lower  
speed, i.e. peripheral  
speed

3) word / half word or  
byte accessible

3) word accessible



④ Monika - 56352

⇒ Mask/Lock feature to avoid accidental changes in GPIO pins.

④ No Mask/Lock feature

Que. 6

What is the difference between Von Neumann & Harvard architecture?

⇒

Von-Neumann architecture

Harvard architecture

1) Address & data bus is common for program memory (ROM) & data memory (RAM).

1) Separate address & data buses for program memory (ROM) & data memory (RAM).

2) Instruction & data cannot be fetched simultaneously.

2) Instruction & data can be fetched simultaneously.

3) op-code & operands are not fetched in single cycle. So slower execution.

3) op-code & operands are fetched in single cycle. So faster execution.

4) Unified cache (i.e. common cache for instruction & data).

4) Separate cache of instruction & data

5) e.g. x86 archi

5) e.g. AVR



Ques. 3 Explain ARM Cortex-M modes? Explain register banking. Why FIQ is faster than IRQ?

⇒ There are two modes of ARM Cortex-M:

1) Thread mode

2) Handler mode

1) Thread mode :-

- When executing normal app's code, the processor can be either in privileged access level or unprivileged access level. This is controlled by a special register called as "CONTROL".

- Can work in privileged or unprivileged mode

e.g. a system can contain an embedded OS kernel that executes in privileged access level & application tasks which execute in unprivileged access level.

In this way, we can set up memory access permissions using the Memory Protection Unit (MPU) to prevent an app's task from corrupting memory & peripherals used by OS kernel & the other tasks.

2) Handler mode :-

When executing an exception, handler such as Interrupt Service Routine (ISR).



⑥

Manika-56352

Date / /

When in privilege handler mode, the processor may always has privileged access level.

### Register banking :-

The registers are grouped together called as register bank.

- 1) General purpose registers (R0-R12)
- 2) Stack pointer (SP) (R13)
- 3) Link register (LR) (R14)
- 4) Program counter (PC) (R15)
- 5) Program status Registers (PSR)
  - Application PSR (APSR)
  - Execution PSR (EPSR)
  - Interrupt PSR (IPSR)

### 1) General purpose registers :-

R0-R7 → Low registers  
16 bit instructions can only access low registers.

R8-R12 → High registers  
can be used with 32-bit instr's & few with 16 bit instr's.  
Initial value of R0 & R12 are undefined.



⑦ Monika-56352

2) Stack pointer (SP) :- (R13)

- used for accessing the stack memory via push & pop operations.

There are two different stack pointers:

1) Main stack pointer (MSP)

- default stack pointer

2) Process stack pointer (PSP)

- used only in Thread mode

3) Link register (R14) (LR) :-

- used for holding the return address when calling a function or subroutine.

4) Program Counter (PC) (R15) :-

- returns the address of the next instruction to be fetched

5) Program Status Registers (PSR) :-

PSR composed of three status reg's:

- Application PSR (APSR)
- Execution PSR (EPSR)
- Interrupt PSR (IPSR)

\* PRIMASK, FAULTMASK & BASEPRI registers :-

- used for exception or interrupt masking
- Each exception has a priority level where smaller no. is a higher priority &



⑧

Monika-56352

larger no. is a lower priority.

PRIMASK :-

when set, it blocks all exceptions except NMI

FAULTMASK :-

similar to PRIMASK, also blocks the HardFault exception.

BASEPRI :-

— blocks exceptions that have same or lower priority level, while allowing exceptions with a priority level higher to be accepted by the processor.

\* Control Register :-

— Handle access level & stack

IRQ vs FIQ

- FIQ has higher priority than IRQ exception. If both arises at the same time, FIQ-Handler will be executed first.
- IRQ-Handler must have save execution context before actual exception handling. FIQ-Handler may use dedicated R8-R12 registers for its computing. It may skip execution context saving.
- FIQ vector address is last in exception vector table. FIQ-Handler can be directly placed on that address (0x1c) instead of additional jump(B) instr<sup>n</sup>. This is not possible of IRQ-Handler.



Monika - 56352

Q. 3) What is pipeline? What are limitations?  
Explain pipeline in ARM architecture.

⇒ Pipelining is the process of accumulating instruction from the processor through a pipeline. It allows storing & executing ~~exe~~ instructions in an orderly process. It is also known as pipeline processing.

Pipelining is a technique where multiple instr's are overlapped during execution. Pipeline is divided into stages & these stages are connected with one another to form a pipe like structure. Instr's enter from one end & exit from another end.

Limitations :-

Pipeline increases the overall instruction throughput.

- 1) Timing variations
- 2) Data hazards
- 3) Branching (Control hazard)
- 4) Interrupts
- 5) Data dependency

Pipeline in ARM :-

| Cycle | Fetch | Decode | Execute |
|-------|-------|--------|---------|
| 1     | A     |        |         |
| 2     | B     | A      |         |
| 3     | C     | B      | A       |



10

Monika-56352

- Fetch loads instr<sup>n</sup> from memory.
- Decode identifies the instr<sup>n</sup> to be executed.
- Execute processes the instr<sup>n</sup> & writes the result back to the register.
- ARM7 has 3 stage pipeline.
- It can complete its process in 3 cycles.
- It has the basic F&E cycle leading to optimum throughput.
- This is why the ARM7 has the lowest throughput as compared to that of its other family members.
- It processes 32 bit data.

Que.4) What is ARM7 pipeline? What are limitations of pipeline?

→ ARM7 has 3 stage pipeline that increases instr<sup>n</sup> flow through the processor upto 3 times. So each instr<sup>n</sup> is executed in three stages:

- 1) Fetch - instr<sup>n</sup> is fetched from memory
- 2) Decode - instr<sup>n</sup> is decoded
- 3) Execute - instr<sup>n</sup> is executed

|                     | Cycle 1 | Cycle 2 | Cycle 3 | Cycle 4 | Cycle 5 |
|---------------------|---------|---------|---------|---------|---------|
| Inst <sup>n</sup> 1 | Fetch   | Decode  | Execute |         |         |
| Inst <sup>n</sup> 2 |         | Fetch   | Decode  | Execute |         |
| Inst <sup>n</sup> 3 |         |         | Fetch   | Decode  | Execute |



Limitations :-

- Designing of the pipelined processor is complex.
- Instrn' latency increases in pipelined processors.
- The throughput of a pipelined processor is difficult to predict.

Que. 8) Explain protocols in detail :  
RS232, T2C, SPI, PS2.

⇒ RS232:

• Physical char :

- 1) Type : Full duplex  
— Peer to peer protocol
- 2) Connections : Three wires  
Tx : data transmission  
Rx : data receive  
GND : Ground

3) Voltage levels

- Follows CMOS voltage levels -

i.e. +3V to +25V → Logic 0

-3V to -25V → Logic 1

4) Baud rate :-

Standard : 9600, 38400, 115200



(12)

Monika - 56352

## Logical characteristics :-

### 1) Data frame :

START + Data + Parity bit + STOP

START bit  $\rightarrow$  always 0

Data bits  $\rightarrow$  5 to 9 bits  $\rightarrow$  LSB first sent

Parity bit  $\rightarrow$  even/odd/no parity bit

STOP bit(s)  $\rightarrow$  always 1

### 2) Error collision conditions :-

#### • Read overrun :-

The microcontroller is processing some data & not read the data transmitted

#### • Write collision :-

The microcontroller is writing next data for transmission before old data is transmitted.

#### • Noise error

## SPI Protocol :-

- High speed protocol
- developed by Motorola for short dist comm (within PCB).



Physical characteristics :-

- Logical characteristics :-

- 13



⑭ Manika - 56352.

3) Mode fault  
- When a SPI device is in master mode & other SPI device force it to become slave

4) Slave abort :-  
If SPI master is communicating with a SPI slave & that slave stop responding suddenly, it

5) CRC error :-  
If CRC (sum of bits) is not matching

### I2C Bus protocol :-

- developed by Philips
- synchronous protocol

### Physical characteristics :-

Type : Bus protocol

- Half duplex
- follows master slave

Wires/connectivity :

- 2 wires protocol
- SDA - Serial data
- SCL - Serial Clock

Voltage level :

TTL voltage level

Frequency :

- 100 kHz - standard mode
- 400 kHz - Fast mode
- 1 MHz - Fast plus mode



Q5

Monika - 56352

Date

Logical char :-

Data frame :-

START + 8 data bits + Ack + STOP

Address frame :-

7 I2caddr bits + Read/Write + Ack

Ques. 10) Which programmer you have used for ARM?

⇒ We have used ST-Link as hardware programmer & STM32 Cube as software program.

Ques. 11) What are ARM processor states?

⇒ ARM processors have two operation states

i) Debug state :-

When the processor is halted (e.g. by the debugger, or after hitting a breakpoint), it enters debugger state & stops executing instr's.

ii) Thumb state

If the processor is running program code (Thumb instr's), it is in Thumb state. Unlike ARM7, there is no ARM state because the



(16)

Monika-56352

Cortex-M processors do not have support the ARM instr<sup>n</sup> set.

Que.15) Explain the instruction set for ARM in detail.

⇒ 1) Data transfer instructions

- `mov rd, #k` (move #k in rd)  
 $rd \Rightarrow$  destination reg<sup>r</sup>  
 $\#k \Rightarrow$  data
- `mvn rd, #k`  
 $\Rightarrow rd = \sim(k)$
- `mrs rd, SFR`  
 [move data from SFR to rd]
- `msr SFR, rs`  
 [move data from rs to SFR]

2) Arithmetic instruction :-

- `add rd, rs`  $\Rightarrow rd = rd + rs$
- `sub rd, rm, rn`  $\Rightarrow rd = rm - rn$
- `subs rd, rm, rn`  $\Rightarrow rd = rm - rn$   
 $\hookrightarrow$  ALU flag updated (NZCV)
- `mul rd, rm, rn`
- `mlo rd, rm, rn, ra`  
 $\Rightarrow rd = rm \times rn + ra$



Monika-56352

3) Logical instructions :-

- `and rd, rn, rm`
- `bic rd, rn, rm`  $\Rightarrow$  `rd = rn & ~rm`;  
 $\Rightarrow$  bit clear
- `tst rm, rn`  $\Rightarrow$  tests nth bit
- `orr rd, rn, rm`  $\Rightarrow$  set nth bit  
`rd = rn | rm`
- `orn rd, rn, rm`  $\Rightarrow$  `rd = rn | ~rm`
- `eor rd, rn, rm`  $\Rightarrow$  Toggle nth bit  
`rd = rn ^ rm`

4) Conditional branching :-

- `cmp rm, rn`  $\Rightarrow$  `rm - rn`, update ALU flags
- `bxx label`

Branching based on cond<sup>n</sup>  $\Rightarrow$ xx  $\rightarrow$  EQ / NE / GT / LE / LT / GE

Q 18) What are the types of addressing modes in ARM?

1) Immediate addressing mode :-

`mov r0, #12`

2) Register addressing mode

`mov r0, r1`

3) Direct addressing mode

`ldr r1, =num1`



- 4) Indirect addressing mode:  
`ldr r0, [r7]`
- 5) Post-indexing, add. mode:  
`ldr r0, [r7], #4`
- 6) Pre-indexing (with immediate offset):  
`ldr r0, [r7, #4]`
- 7) Pre-indexing with auto update  
`ldr r0, [r7, #4]!`
- 8) Post-indexing  
`ldr r0, [r1], #4`
- 9) Pre-indexing with register offset  
`ldr r0, [r1, r2]`
- 10) Pre-indexing with scaling  
`ldr r0, [r1, r2, lsl #4]`
- 11) Program counter relative  
`ldr r0, [PC, #offset]`

Que. 20) What is single issue multiple data (SIMD) processing?

⇒ SIMD processing, in which single instruction is applied on multiple data, is suitable for multimedia processing & therefore it is implemented in contemporary processors. Single instruction multiple data (SIMD), as the name suggests, takes an operation specified in one instruction & applies it to more than one set of data elements at the same time. For example, in a traditional scalar microprocessor, an add



(19)

Monika-56352

Date / /

operation would add together a single pair of operands & produce a single result. In SIMD processing, a no. of independent operand pairs are added together to produce the same no. of independent sums.

|             |  |             |             |             |             |
|-------------|--|-------------|-------------|-------------|-------------|
| $x_0$       |  | $x_3$       | $x_2$       | $x_1$       | $x_0$       |
| +           |  | +           | +           | +           | +           |
| $y_0$       |  | $y_3$       | $y_2$       | $y_1$       | $y_0$       |
| +           |  | =           | =           | =           | =           |
| $x_0 + y_0$ |  | $x_3 + y_3$ | $x_2 + y_2$ | $x_1 + y_1$ | $x_0 + y_0$ |

e2) What is the use of 'SWI' in ARM assembly?

⇒ Soft Interrupt (SWI) functions are functions that run in Supervisor mode of ARM7 & ARM9 core & are interrupt protected.

SWI functions can accept arguments & can return values. They are used in the same way as other functions.

It generates different code instructions to call SWI functions. SWI functions are called by executing the SWI instruction. When executing the SWI instruction, the controller changes the running mode to a Supervisor mode & blocks any further TR@ interrupt requests. The FI@ interrupts are not disabled in this mode. When the ARM controller leaves this mode, interrupts



(20)

Monika-56352

are enabled again.

SWI functions 0-7 are reserved for the RTX kernel.

SWI functions can still be interrupted by FIQ interrupts.

Que. 23) Tell about the Exception handling in ARM processor. What does the ARM core do automatically for every exception?

⇒ An exception is any condition that needs to halt the normal sequential execution of instructions. Exception handling is the method of processing these exceptions.

ARM processor exceptions & associated modes :

| <u>Exception</u>               | <u>Mode</u> | <u>Main purpose</u>                              |
|--------------------------------|-------------|--------------------------------------------------|
| 1) Fast Interrupt Request      | FIQ         | fast interrupt request handling                  |
| 2) Interrupt request           | IRQ         | Interrupt request handling                       |
| 3) SWI & Reset                 | SVC         | Protected mode for operating systems             |
| 4) Prefetch abort & data abort | abort       | virtual memory and/or memory protection handling |



(21)

Monika-56352

Page No.

Date

|                          |       |                                             |
|--------------------------|-------|---------------------------------------------|
| 5) undefined instruction | Undef | Software emulation of hardware coprocessors |
|--------------------------|-------|---------------------------------------------|

Each exception causes the core to enter a specific mode. In addition, any of the ARM processor modes can be entered manually by changing the cpsr. User & system mode are the only two modes that are not entered by a corresponding exception, in other words, to enter these modes, you must modify the cpsr.

When an exception causes a mode change, the core automatically

- saves the cpsr to the spsr of the exception mode
- saves the pc to the lr of the exception mode
- sets the cpsr to the exception mode
- sets pc to the address of exception handler.

When an exception occurs, the ARM processor always switches to ARM state.



22

Monika-56352

Que 24) Explain memory map in ARM architecture.

→ In the 4 GB addressable memory space, some parts are allocated to internal peripherals within the processor such as the NVIC & debug components. The memory locations of these internal components are fixed.

This arrangement allows:

- The processor design to support different types of memories & devices out of the box.
- Optimised arrangement for higher performance.

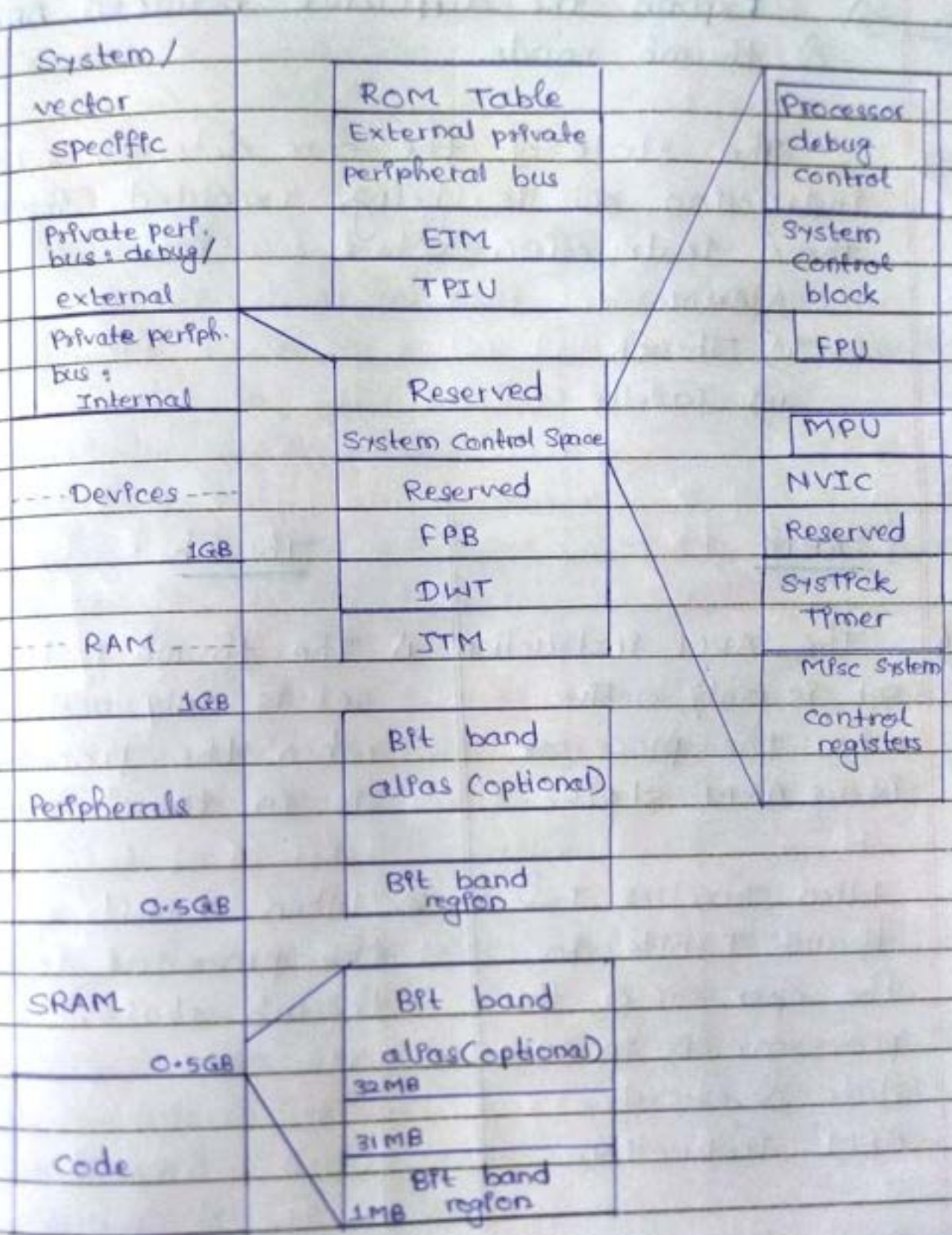
While it is possible to store & execute program code in SRAM & RAM regions, the processor design is not optimized for such operation & requires one extra clock cycle per instruction for each instruction fetch.

As a result, the performance is slightly slower when executing program codes through the system bus.

Program execution from peripherals, devices & system memory regions is not allowed.



23





(24)

Monika. 56352

Que. 25) Explain the difference between ARM & thumb mode.

⇒ The state of the core determines which instruction set is being executed. There are three instruction sets:

i) ARM

ii) Thumb

iii) Jazelle

### ARM

i) The ARM instruction set is only active when the processor is in ARM state.

2) When Jazelle & thumb T bits in the cpsr are 0, the processor is in ARM state & executes ARM instructions.

3) Instruction size  
= 32 bit

4) Core instructions  
= 58

### Thumb

i) The thumb instruction set is only active when the processor is in thumb state.

2) When T bit = 1, the processor is in thumb state.

16 bit

= 30



- |                                 |                                     |                                                       |
|---------------------------------|-------------------------------------|-------------------------------------------------------|
| → Condition execution           | - most                              | - only branch instr's                                 |
| 2) Data processing instructions | - Access to barrel shifter & ALU    | - separate barrel shifter & ALU instructions          |
| 3) Program status register      | - read-write in privileged mode     | - no direct access                                    |
| 4) Register usage               | - 15 general purpose registers + pc | - 8 general purpose registers + 7 high registers + pc |

Ques. 26) Explain about the bits in CPSR register.

→ The ARM core uses the cpsr to monitor & control internal operations. The cpsr is a dedicated 32-bit register & resides in the register file.

Fig. shows the basic layout of a generic program status register. The shaded parts are reserved for future expansion.

The cpsr is divided into four fields, each 8 bits wide:



(26)

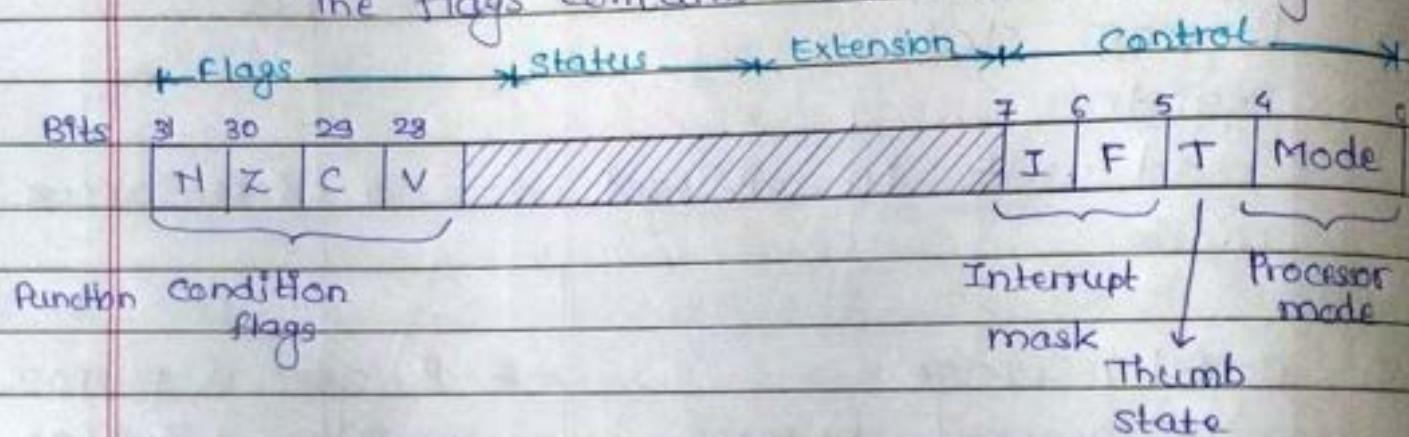
Monika 56352

- flags
- status
- extension
- control

In current design, the extension & status fields are reserved for future use.

The control field contains the processor mode, state and interrupt mask bits.

The flags contains the condition flags.



| Flag | Flag Name | Set when                                                 |
|------|-----------|----------------------------------------------------------|
| N    | Negative  | Bit 31 of the result is a binary 1                       |
| Z    | Zero      | The result is zero, frequently used to indicate equality |
| C    | Carry     | The result causes an unsigned carry                      |
| V    | overflow  | The result causes a signed overflow                      |

I  $\Rightarrow$  masks IRQ when set to binary 1  
 F  $\Rightarrow$  masks FIQ when set to binary 1



Monika - 56352

## Processor mode

| Mode                     | Abbreviation | Privileged | Mode [4:0] |
|--------------------------|--------------|------------|------------|
| • Abort                  | abt          | Yes        | 1011       |
| • Fast Interrupt Request | fiq          | Yes        | 10001      |
| • Interrupt request      | irq          | Yes        | 10010      |
| • Supervisor             | svc          | Yes        | 10011      |
| • System                 | sys          | Yes        | 11111      |
| • Undefined              | und          | Yes        | 11011      |
| • User                   | usr          | No         | 10000      |

Que-27) What is an interrupt? Write a C program to handle external interrupt for an ARM chip.

⇒ Interrupt is a signal generated by a hardware or software when a process or an event needs immediate attention.

It alerts the processor to a high-priority process requiring interruptions of the current working process.



Program :-

switch.h

~~##~~ ifndef EXTI\_H\_

# define EXTI\_H\_

# include "stm32f4xx.h"

# define SWITCH\_EXTI 0

# define SWITCH\_EXTI\_IRQn EXTI0\_IRQn

# define SWITCH\_CLK\_EN

RCC\_AHB1ENR\_GPIODEN\_PIN

# define SWITCH\_GPIO GPIOA

# define SWITCH\_PIN 0

// switch EXTIOR[3:0] = 0000

# define SWITCH\_EXTICR SYSEFQ →  
EXTICR[0]

# define SWITCH\_EXTICR\_MSK

(1 BV(3) | BV(2) | BV(0) | BV(1))

void switchExtInit(void);

# end if